操作系统

进程同步与互斥

邓一星 2025.3.18

3.1 进程的同步与互斥

同步与互斥的引入

- 同步：指多个进程中发生的事件存在着某种时序关系，它们必须按规定时序执行，以共同完成一项任务 。
- 互斥：多个进程不能同时使用同一资源。

临界资源实例

例：P1，P2两进程共享变量COUNT（COUNT的初值为5）

```
P1:{
  R1=COUNT;
  R1=R1+1;
  COUNT=R1; }
```

```
P2:{
  R2=COUNT;
  R2=R2+1;
  COUNT=R2; }
```

临界区

P1:{

进入区

R1=COUNT;

R1=R1+1;

COUNT=R1;

退出区

}

P2:{

进入区

R2=COUNT;

R2=R2+1;

COUNT=R2;

退出区

}

1)执行顺序 P1-->P2

2)执行顺序

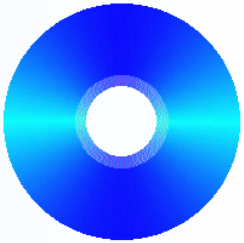
P1：{R1=COUNT}

P2：{R2=COUNT}

P1：{R1=R1+1；
COUNT=R1}

P2：{R2=R2=1；
COUNT=R2}

同步机制应遵循的准则



访问临界资源的进程描述为

While(1){

进入区

临界区

退出区

剩余区

}

- 空闲让进

  当无进程处于临界区时，临界资源处于空闲状态。此时允许进程进入临界区。

- 忙则等待

  当已有进程进入临界区时，临界资源正在被访问，其他想进入临界区的进程必须等待。

同步机制应遵循的准则

访问临界资源的进程描述为

While(1){

进入区

临界区

退出区

剩余区

}

- 有限等待

  对于要求访问临界资源的进程，应保证在有效的时间内进入，以免进入“死等”状态。

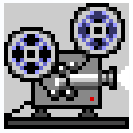
- 让权等待

  当进程不能进入临界区时，应立即释放处理机，以免进程进入“忙等”。

互斥实现的硬件方法

- 禁止中断
- 专用机器指令
  - TS（Test and Set）指令
  - Swap指令

TS（Test and Set）指令



```
//TS指令：
    boolean  TS(lock);
    boolean  lock;
    {    boolean  temp;
         temp = lock;
         lock = true;
         return  temp;}
```

```
//TS指令的使用
    while (TS(lock))
          /*什么也不做*/;
    临界区;
    lock = false;
    剩余区;
```

- 为了实现互斥，设布尔变量lock，其初值为false,表示资源空闲。利用TS指令实现互斥。

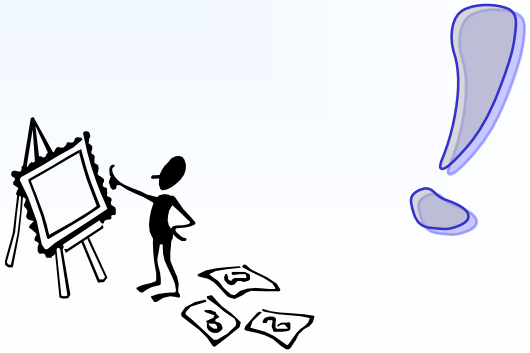
用软件方法解决互斥问题

——单标志算法

```
//进程0
while (turn!=0)
        //什么都不做;

临界区;
turn = 1;
剩余区;
```

```
//进程1
while (turn!=1) do
        //什么都不做 ‘;
临界区;
turn = 0;
剩余区;
```

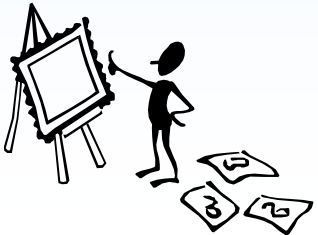


用软件方法解决互斥问题

——双标志、先检查算法

```
//进程0
    while (flag[1])
        //什么都不做；
    flag[0]=true;
    临界区;
    flag[0] =false;
    剩余区;
```

```
//进程1
    while ( flag[0])
        //什么都不做；
    flag[1]=true;
    临界区;
    flag[1] =false;
    剩余区;
```

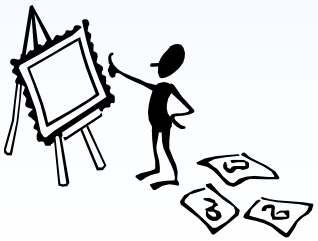


用软件方法解决互斥问题

——双标志、先修改后检查算法

```
//进程0
    flag[0]=true;
    while (flag[1])
        //什么也不做;
    临界区;
    flag[0] =false;
    剩余区;
```

```
//进程1
    flag[1]=true;
    while (flag[0])
        //什么也不做;
    临界区;
    flag[1] =false ;
    剩余区;
```

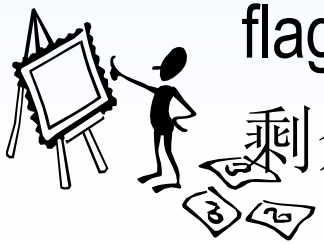


用软件方法解决互斥问题

——先修改、后检查、后修改算法

```
//进程0
    flag[0]=true;
    turn=1;
    while (flag[1]) &&
    (turn==1)
        //什么也不做;
    临界区;
    flag[0] =false ;
    剩余区;
```

```
//进程1
    flag[1]=true;
    turn=0;
    while (flag[0]) &&
    (turn==0)
        //什么也不做;
    临界区;
    flag[1] =false ;
    剩余区;
```



信号量和PV操作

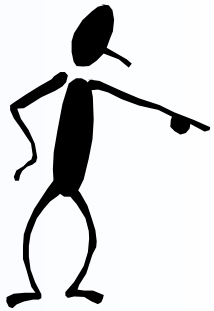
- 1965年，荷兰学者Dijkstra提出了信号量机制，卓有成效地解决了进程同步问题。

- 记录型信号量的定义

```
struct semaphore{
  int value;
  struct PCB *queue;
}
```

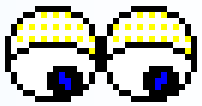
信号灯的PV操作

```
void  wait(semaphore  s)
{   s.value = s.value - 1;
    if  (s.value < 0)
      block(s.queue);
    /* 将进程阻塞，并将其投入等待队列s.queue */
}
```



```
void  signal(semaphore  s)
{   s.value = s.value + 1;
    if  (s.value <= 0)
      wackup(s.queue);
    /* 唤醒阻塞进程，将其从等待队列s.queue 取出，投入就绪队列*/
}
```

信号灯的物理意义



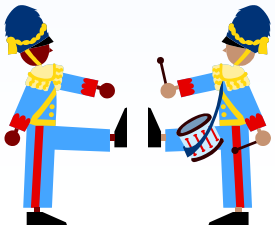
从资源的观点看信号灯的意义:

- s.value的初值表示系统中某种资源数目。
- wait(s)表示要申请一个资源。
- signal(s)表示要释放一个资源。
- s.value <0时，|s.value|表示等待队列的进程数。

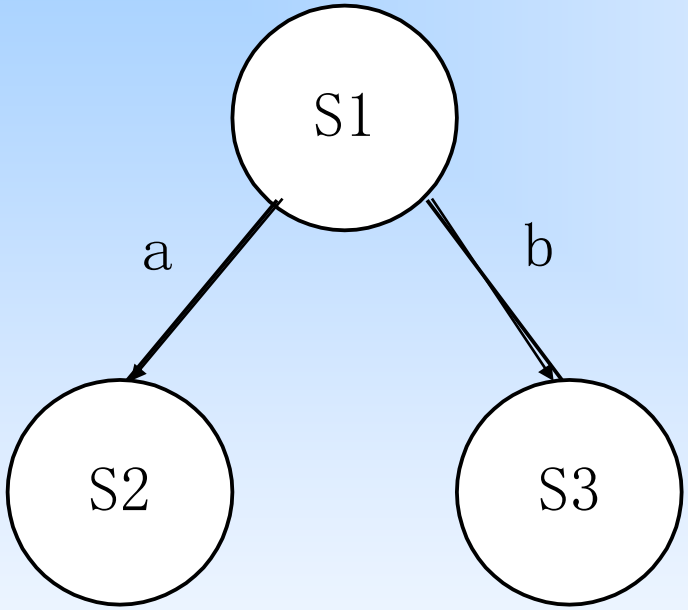
用信号灯解决互斥问题

```
semaphore mutex=1;
P1:
    while (1){
        P(mutex);
        临界区;
        V(mutex);
        剩余区;
    };
```

```
P2:
    while (1){
        P(mutex);
        临界区
        V(mutex);
        剩余区;
    };
```



用信号灯解决同步问题

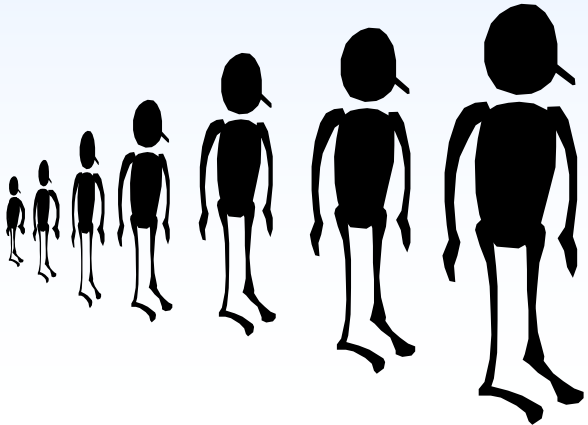


semaphore a,b=0,0;

{s1; V(a); V(b)}

{P(a); s2}

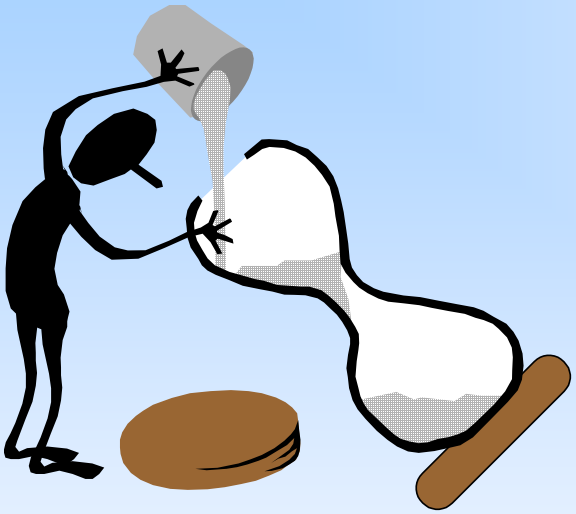
{P(b); s3}

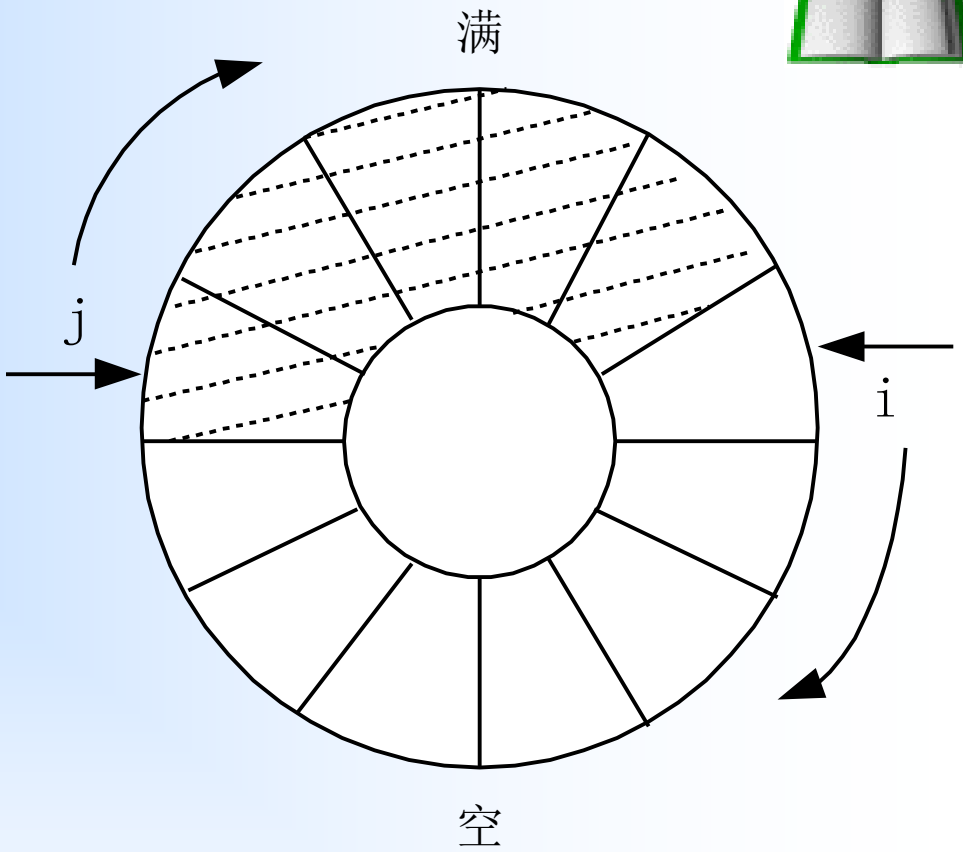


3.2经典进程同步问题

- 生产者——消费者问题
- 读者——写者问题
- 哲学家进餐问题
- 打磕睡的理发师问题

生产者-消费者问题





同步问题：缓冲池满不能生产，缓冲池空不能消费

互斥问题：缓冲池访问需互斥

用信号量解决“生产者-消费者”问题

```
int  i,j;
ITEM  buffer[n];
ITEM  data_p, data_c;
```

```
void  producer() //生产者进程
 {while  (true)
    {produce an item in data_p;
          buffer[i] = data_p;
    i = (i + 1) % n;
    }
 }
```

```
void consumer()//消费者进程
 {while  (true)
  {      data_c =  buffer[j];
   j = (j + 1) % n;
          consume  the  item  in
data_c;}
  }
```

用信号量解决“生产者-消费者”问题

```
semaphore  empty = n;
semaphore  full = 0;

int  i,j;
ITEM  buffer[n];
ITEM  data_p, data_c;

void  producer() //生产者进程
  {while  (true)
     {produce an item in data_p;
      P(empty);
         buffer[i] = data_p;
      i = (i + 1) % n;
           V(full);}
   }

void consumer()//消费者进程
 {while  (true)
  { P(full);
         data_c =  buffer[j];
    j = (j + 1) % n;
         V(empty);
    consume the item in data_c
  }
```

用信号量解决“生产者-消费者”问题

```
semaphore  mutex =1;
semaphore  empty = n;
semaphore  full = 0;

void  producer() //生产者进程
  {while  (true)
     {produce an item in data_p;
      P(empty);
      P(mutex);
     生产...
      V(mutex);
      V(full);}
   }
```

```
void  consumer()//消费者进程
{while  (true)
  { P(full);
    P(mutex);
    消费..
    V(mutex);
    V(empty);
    consume the item in data_c;}
  }
```

读者-写者问题

- 读数据对象的进程称为“读者”，写数据对象的进程称为“写者”
- 问题描述：
  - 读者可同时读；
  - 读者读时，写者不可写；
  - 写者写时，其他的读者、写者均不可进入。

用信号量解决读者-写者问题

读者与写者互斥

semaphore Wmutex=1;

读者进程：

```
while(true)
{
   P(Wmutex);
   读;
   V(Wmutex);   }
```

写者进程：

```
while(true)
{
   P(Wmutex);
   写;
   V(Wmutex);
}
```

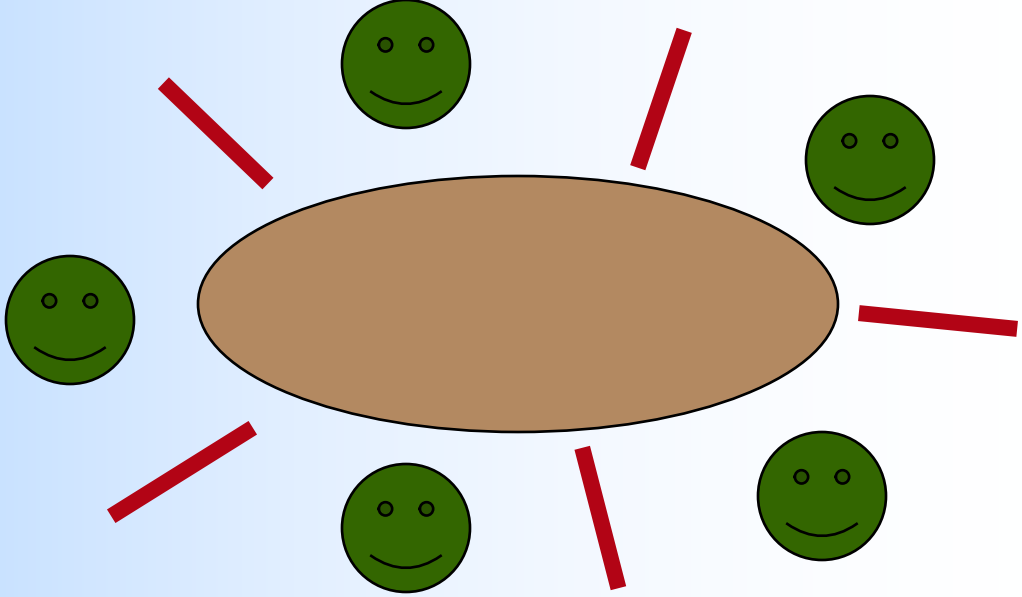
用信号量解决读者-写者问题

```
Semaphore Wmutex=1; int Rcount;
void  reader()    {while  (true)
  {
        if  (Rcount == 0)    P(Wmutex);
        Rcount = Rcount + 1;
        read;
        Rcount = Rcount - 1;
        if  (Rcount == 0)    V(Wmutex);
          }}
```

用信号量解决读者-写者问题

```
Semaphore Wmutex,Rmutex=1,1; int Rcount;
void reader()    {while (true)
  { P(Rmutex);
        if (Rcount == 0)    P(Wmutex);
        Rcount = Rcount + 1;
  V(Rmutex);
  read;
  P(Rmutex);
        Rcount = Rcount - 1;
        if (Rcount == 0)    V(Wmutex);
        V(Rmutex);    }}
```

哲学家进餐问题



- 这里的问题是哲学家只有拿到靠近他的两支筷子才能进餐，而拿到两支筷子的条件是他的左、右邻居此时都没有进餐。

用信号量解决哲学家进餐问题

```
semaphore  chopstick[5] ={1,1,1,1,1};
void  philosopher (int i )    /*哲学家进程*/
{while  (true)
    { P(chopstick[i]);
      P(chopstick[(i + 1) % 5]);
      eating;      /* 进餐 */
     V(chopstick[i]);
     V(chopstick[(i + 1) % 5]);
      thinking;      /* 思考 */}
}
```

3.3 AND信号量

Swait(s1,s2,…,sn)

Ssignal(s1,s2,…,sn)

一次申请或释放多种信号量！

AND信号量定义

```
Swait(s1,s2,…,sn)
{if (s1 >= 1 && s2 >= 1 && … && sn>= 1)
      { /* 满足资源要求时*/
   for (i = 1; i <= n; i = i + 1)
         si = si -1;  }
  else
      { /* 某些资源不能满足要求时*/
   block(si.queue ) /*将进程投入第一个小于1的信号量的等待队列si.queue */ ;
   }
}
```

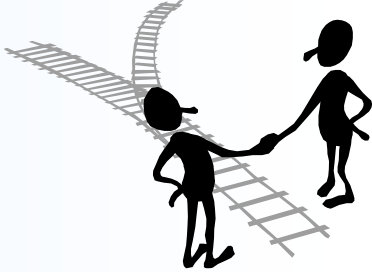
AND信号量定义

```
Ssignal(s1,s2,…,sn)
{  for (i = 1; i <= n; i = i + 1)
        {si = si +1;
        for (  等待队列si.queue中的每个进程P)
                if (  进程P通过Swait中的测试)
                        {/*  通过检查，即资源够用  */
                        wackup(p);    //唤醒进程P；  }
         else
                        {/*  未通过检查，即资源不够用  */
                        进程P继续等待；  }
        }
}
```

用AND信号量解决哲学家进餐问题

```
semaphore  chopstick[5] ={1,1,1,1,1};
void  philosopher ( int i )    /*哲学家进程*/
{while  (true)
        {Swait(chopstick[i], chopstick[ (i + 1) % 5]);
         eat();      /* 进餐 */
         Ssignal(chopstick[i], chopstick[(i + 1) % 5]);
         think();      /* 思考 */
        }
}
```

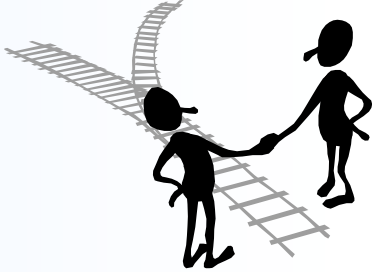
3.5 进程通信



信号灯作为进程同步和互斥工具是卓有成效的。但作为通信工具就不够理想。其原因为：

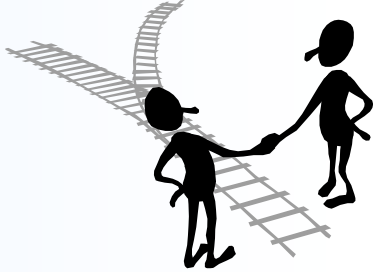
- 效率低——一次只传一条消息。
- 通信对用户不透明。

进程通信的类型



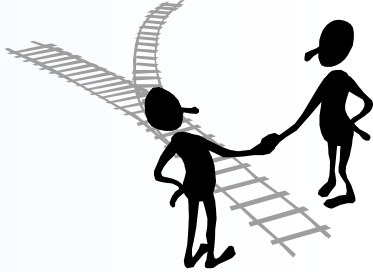
- 共享存储器系统
- 消息传递系统
  - 直接通信方式
  - 间接通信方式
- 管道通信

进程通信中的几个问题



- 通信链路的建立方式
  - 显示建立链路
  - 隐式建立链路
- 通信方向
- 通信链路连接方式

进程通信中的几个问题



- 通信链路的容量
- 数据格式
- 同步方式
  - 阻塞方式
  - 不阻塞方式

客户—服务器系统通信

常用的通信方式：

- 命名管道
- 套接字
- 远程过程调用